

■ 3.7.12 Advanced Topic: Tensors

Tensors are mathematical objects that give generalizations of vectors and matrices. In *Mathematica*, a tensor is represented as a set of lists, nested to a certain number of levels. The nesting level is the *rank* of the tensor.

rank 0	scalar
rank 1	vector
rank 2	matrix
rank k	rank k tensor

Interpretations of nested lists.

A tensor of rank k is essentially a k -dimensional table of values. To be a true rank k tensor, it must be possible to arrange the elements in the table in a k -dimensional cuboidal array. There can be no holes or protrusions in the cuboid.

The *indices* that specify a particular element in the tensor correspond to the coordinates in the cuboid. The *dimensions* of the tensors correspond to the side lengths of the cuboid.

One simple way that a rank k tensor can arise is in giving a table of values for a function of k variables. In physics, the tensors that occur typically have indices which run over the possible directions in space or spacetime. Notice, however, there is no built-in notion of covariant and contravariant tensor indices in *Mathematica*: you have to set these up explicitly using metric tensors.

<code>Table[f, {i₁, n₁}, {i₂, n₂}, ..., {i_k, n_k}]</code>	create an $n_1 \times n_2 \times \dots \times n_k$ tensor whose elements are the values of f
<code>Array[a, {n₁, n₂, ..., n_k}]</code>	create an $n_1 \times n_2 \times \dots \times n_k$ tensor with elements given by applying a to each set of indices
<code>Dimensions[t]</code>	give a list of the dimensions of a tensor
<code>TensorRank[t]</code>	find the rank of a tensor
<code>MatrixForm[t]</code>	print with the elements of t arranged in a two-dimensional array

Functions for creating and testing the structure of tensors.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Here is a $2 \times 3 \times 2$ tensor.

```
In[1]:= t = Table[i1+i2 i3, {i1, 2}, {i2, 3}, {i3, 2}]
Out[1]= {{{2, 3}, {3, 5}, {4, 7}},
          {{3, 4}, {4, 6}, {5, 8}}}
```

This is another way to produce the same tensor.

```
In[2]:= Array[(#1 + #2 #3)&, {2, 3, 2}]
Out[2]= {{{2, 3}, {3, 5}, {4, 7}},
          {{3, 4}, {4, 6}, {5, 8}}}
```

MatrixForm displays the elements of the tensor in a two-dimensional array. You can think of the array as consisting of two 3×2 matrices arranged side by side.

```
In[3]:= MatrixForm[ t ]
          2 3 3 4
          3 5 4 6
Out[3]//MatrixForm= 4 7 5 8
```

Dimensions gives the dimensions of the tensor.

```
In[4]:= Dimensions[ t ]
Out[4]= {2, 3, 2}
```

Here is the 111 element of the tensor.

```
In[5]:= t[[1, 1, 1]]
Out[5]= 2
```

TensorRank gives the rank of the tensor.

```
In[6]:= TensorRank[ t ]
Out[6]= 3
```

The rank of a tensor is equal to the number of indices needed to specify each element. You can pick out subtensors by using a small number of indices.

Transpose [<i>t</i>]	transpose the first two indices in a tensor
Transpose [<i>t</i> , { <i>p</i> ₁ , <i>p</i> ₂ , ...}]	transpose the indices in a tensor so that the <i>p_k</i> th becomes the <i>k</i> th
Outer [<i>f</i> , <i>t</i> ₁ , <i>t</i> ₂ , ...]	form the generalized outer product of the tensors <i>t_i</i> with “multiplication operator” <i>f</i>
<i>t</i> ₁ . <i>t</i> ₂	form the dot product of <i>t</i> ₁ and <i>t</i> ₂ (last index of <i>t</i> ₁ contracted with first index of <i>t</i> ₂)
Inner [<i>f</i> , <i>t</i> ₁ , <i>t</i> ₂ , <i>g</i>]	form the generalized inner product, with “multiplication operator” <i>f</i> and “addition operator” <i>g</i>

Tensor manipulation operations.

You can think of a rank *k* tensor as having *k* “slots” into which you insert

indices. Applying **Transpose** is effectively a way of reordering these slots. If you think of the elements of a tensor as forming a k -dimensional cuboid, you can view **Transpose** as effectively rotating (and possibly reflecting) the cuboid.

In the most general case, **Transpose** allows you to specify an arbitrary reordering to apply to the indices of a tensor. The function **Transpose** $[T, \{p_1, p_2, \dots, p_k\}]$, gives you a new tensor T' such that the value of $T'_{i_1 i_2 \dots i_k}$ is given by $T_{i_{p_1} i_{p_2} \dots i_{p_k}}$.

If you originally had a $n_1 \times n_2 \times \dots \times n_k$ tensor, then by applying **Transpose**, you will get a $n_{p_1} \times n_{p_2} \times \dots \times n_{p_k}$ tensor.

Here is a matrix, that you can also think of as a 2×3 tensor.

```
In[7]:= m = {{a, b, c}, {ap, bp, cp}}
Out[7]= {{a, b, c}, {ap, bp, cp}}
```

Applying **Transpose** gives you a 3×2 tensor. **Transpose** effectively interchanges the two “slots” for tensor indices.

```
In[8]:= mt = Transpose[m]
Out[8]= {{a, ap}, {b, bp}, {c, cp}}
```

The element $m[[2, 3]]$ in the original tensor becomes the element $m[[3, 2]]$ in the transposed tensor.

```
In[9]:= { m[[2, 3]], mt[[3, 2]] }
Out[9]= {cp, cp}
```

This produces a $2 \times 3 \times 1 \times 2$ tensor.

```
In[10]:= t = Array[a, {2, 3, 1, 2}]
Out[10]= {{{{a[1, 1, 1, 1], a[1, 1, 1, 2]]},
           {{a[1, 2, 1, 1], a[1, 2, 1, 2]]},
           {{a[1, 3, 1, 1], a[1, 3, 1, 2]]}},
          {{{a[2, 1, 1, 1], a[2, 1, 1, 2]]},
           {{a[2, 2, 1, 1], a[2, 2, 1, 2]]},
           {{a[2, 3, 1, 1], a[2, 3, 1, 2]]}}}}
```

This transposes the first two levels of t .

```
In[11]:= tt1 = Transpose[t]
Out[11]= {{{{a[1, 1, 1, 1], a[1, 1, 1, 2]]},
           {{a[2, 1, 1, 1], a[2, 1, 1, 2]]}},
          {{{a[1, 2, 1, 1], a[1, 2, 1, 2]]},
           {{a[2, 2, 1, 1], a[2, 2, 1, 2]]}},
          {{{a[1, 3, 1, 1], a[1, 3, 1, 2]]},
           {{a[2, 3, 1, 1], a[2, 3, 1, 2]]}}}}
```

The result is a $3 \times 2 \times 1 \times 2$ tensor.

```
In[12]:= Dimensions[ tt1 ]
Out[12]= {3, 2, 1, 2}
```

Outer products, and their generalizations, are a way of building higher-rank

tensors from lower-rank ones. Outer products are also sometimes known as direct, tensor or Kronecker products.

From a structural point of view, the tensor you get from `Outer[f, t, u]` has a copy of the structure of u inserted at the “position” of each element in t . The elements in the resulting structure are obtained by combining elements of t and u using the function f .

This gives the “outer f ” of two vectors.
The result is a matrix.

```
In[13]:= Outer[ f, {a, b}, {ap, bp} ]
Out[13]= {{f[a, ap], f[a, bp]}, {f[b, ap], f[b, bp]}}
```

If you take the “outer f ” of a length 3 vector with a length 2 vector, you get a 3×2 matrix.

```
In[14]:= Outer[ f, {a, b, c}, {ap, bp} ]
Out[14]= {{f[a, ap], f[a, bp]}, {f[b, ap], f[b, bp]},
           {f[c, ap], f[c, bp]}}
```

The result of taking the “outer f ” of a 2×2 matrix and a length 3 vector is a $2 \times 2 \times 3$ tensor.

```
In[15]:= Outer[ f, {{m11, m12}, {m21, m22}}, {a, b, c} ]
Out[15]= {{{f[m11, a], f[m11, b], f[m11, c]},
            {f[m12, a], f[m12, b], f[m12, c]}},
           {{f[m21, a], f[m21, b], f[m21, c]},
            {f[m22, a], f[m22, b], f[m22, c]}}}
```

Here are the dimensions of the tensor.

```
In[16]:= Dimensions[ % ]
Out[16]= {2, 2, 3}
```

If you take the generalized outer product of a $m_1 \times m_2 \times \dots \times m_r$ tensor and a $n_1 \times n_2 \times \dots \times n_s$ tensor, you get a $m_1 \times \dots \times m_r \times n_1 \times \dots \times n_s$ tensor. If the original tensors have ranks r and s , your result will be a rank $r + s$ tensor.

In terms of indices, the result of applying `Outer` to two tensors $T_{i_1 i_2 \dots i_r}$ and $U_{j_1 j_2 \dots j_s}$ is the tensor $V_{i_1 i_2 \dots i_r j_1 j_2 \dots j_s}$ with elements $f[T_{i_1 i_2 \dots i_r}, U_{j_1 j_2 \dots j_s}]$.

In doing standard tensor calculations, the most common function f to use in `Outer` is `Times`, corresponding to the standard outer product.

Particularly in doing combinatorial calculations, however, it is often convenient to take f to be `List`. Using `Outer`, you can then get combinations of all possible elements in one tensor, with all possible elements in the other.

In constructing `Outer[f, t, u]` you effectively insert a copy of u at every point in t . To form `Inner[f, t, u]`, you effectively “glue” edges of u on to edges of t . The idea is to take a $m_1 \times m_2 \times \dots \times m_r$ tensor and a $n_1 \times n_2 \times \dots \times n_s$ tensor, with $m_r = n_1$, and get a $m_1 \times m_2 \times \dots \times m_{r-1} \times n_2 \times \dots \times n_s$ tensor as the result.

The simplest examples are with vectors. If you apply `Inner` to two vectors of

equal length, you get a scalar. `Inner[f, v1, v2, g]` gives a generalization of the usual scalar product, with f playing the role of multiplication, and g playing the role of addition.

This gives a generalization of the standard scalar product of two vectors.

```
In[17]:= Inner[f, {a, b, c}, {ap, bp, cp}, g]
Out[17]= g[f[a, ap], f[b, bp], f[c, cp]]
```

This gives a generalization of a matrix product.

```
In[18]:= Inner[f, {{1, 2}, {3, 4}}, {{a, b}, {c, d}}, g]
Out[18]= {{g[f[1, a], f[2, c]], g[f[1, b], f[2, d]]},
          {g[f[3, a], f[4, c]], g[f[3, b], f[4, d]]}}
```

Here is a $3 \times 2 \times 2$ tensor.

```
In[19]:= a = Array[1&, {3, 2, 2}]
Out[19]= {{{1, 1}, {1, 1}}, {{1, 1}, {1, 1}},
          {{1, 1}, {1, 1}}}
```

Here is a $2 \times 3 \times 1$ tensor.

```
In[20]:= b = Array[2&, {2, 3, 1}]
Out[20]= {{{2}, {2}, {2}}, {{2}, {2}, {2}}}
```

This gives a $3 \times 2 \times 3 \times 1$ tensor.

```
In[21]:= a . b
Out[21]= {{{{4}, {4}, {4}}, {{4}, {4}, {4}}},
          {{{4}, {4}, {4}}, {{4}, {4}, {4}}},
          {{{4}, {4}, {4}}, {{4}, {4}, {4}}}}
```

Here are the dimensions of the result.

```
In[22]:= Dimensions[%]
Out[22]= {3, 2, 3, 1}
```

You can think of `Inner` as performing a “contraction” of the last index of one tensor with the first index of another. If you want to perform contractions across other pairs of indices, you can do this by first transposing the appropriate indices into the first or last position, then applying `Inner`, and then transposing the result back.

In many applications of tensors, you need to insert signs to implement anti-symmetry. The function `Signature[{i1, i2, ...}]`, which gives the signature of a permutation, is often useful for this purpose.